

Money Weather API 명세서

기능 명세서 초안과 기획 초안을 기준으로 구현된 Spring Boot 백엔드 API 구성 및 기능 설명

1. 문서 개요

본 문서는 현재 구현된 Money Weather 백엔드의 전체 API, 요청/응답 형태, 기능별 역할, 운영 모니터링 항목을 정리한 개발 명세서입니다. 기본 업무 API 경로는 /api/v1이며, 운영 Actuator 엔드포인트는 /actuator 아래에 노출됩니다.

항목	내용
백엔드	Spring Boot, Spring MVC, Spring Data JPA, Flyway
기본 DB	H2 메모리 DB, postgres 프로필에서 PostgreSQL 사용
인증	개발용 HMAC Bearer 토큰, X-User-Id, userId query 지원
AI	기본 rule-based provider, OpenAI 호환 Chat Completions provider 선택 가능
검증	컨트롤러 통합 테스트 9개, 계산 로직 단위 테스트 5개 통과

2. 기능별 요약

기능 영역	포함 API	주요 내용
인증	POST /api/v1/auth/dev-token	개발 및 테스트용 Bearer 토큰을 발급합니다.
사용자/시드	GET /api/v1/users/me, POST /api/v1/dev/seed	현재 사용자 조회와 데모 데이터 초기화를 제공합니다.
자산 기초 데이터	GET /accounts, GET /cards, GET /categories	계좌 잔액, 카드 결제일, 거래 분류 기준을 제공합니다.
거래 관리	GET /transactions, PATCH /transactions/{transactionId}/category	월별 거래 검색과 카테고리 보정을 지원합니다.
예정 금융 이벤트	GET/POST/PATCH/DELETE /financial-events	카드 결제, 급여, 구독료 등 미래 현금흐름을 관리합니다.
반복 규칙	POST/GET/PATCH/DELETE /recurring-rules	월 반복 입출금 규칙을 생성, 조회, 수정, 비활성화합니다.
예측/대시보드	GET /forecasts, GET /dashboard, POST /available-funds/simulations	예상 잔액, 자금 날씨, 최저 잔액일, 추가 지출 영향을 계산합니다.
AI Agent	/agent/* 8개 API	대화 저장, 답변, 추천 질문, 소비 분석, 반복 지출 탐지, 행동 추천, 근거 조회를 제공합니다.
예산/안내/알림	GET/PUT /budgets/monthly, GET /weather-statuses, GET /forecast-alerts	월 예산, 날씨 상태 안내, 위험 알림을 제공합니다.
운영/모니터링	/actuator/health, /metrics, /prometheus, api_request_logs	상태 확인, 메트릭, Prometheus 연동, 요청 로그 저장을 지원합니다.

3. 전체 API 색인

아래 표는 프론트엔드 연동과 QA 체크리스트로 사용할 수 있는 전체 엔드포인트 목록입니다.

Method	Path	영역	목적	요청	응답
POST	/api/v1/auth/dev-token	인증	개발용 Bearer 토큰 발급	body: userId	tokenType, accessToken
GET	/api/v1/users/me	사용자	현재 사용자 정보 조회	query: userId optional	userId, name, baseMonth, status
POST	/api/v1/dev/seed	개발	시드 데이터 생성/초기화	body: userId, baseMonth, reset	seeded, userId, baseMonth
GET	/api/v1/accounts	자산	계좌 목록과 총 잔액 조회	query: includedOnly	totalBalance, accounts
GET	/api/v1/cards	자산	카드 목록 조회	query: active optional	cards
GET	/api/v1/categories	분류	거래 카테고리 조회	query: categoryType optional	categories
GET	/api/v1/transactions	거래	월별 거래 검색	query: month, transactionType, categoryId, keyword, page, size	items, page, size, totalElements
PATCH	/api/v1/transactions/{transactionId}/category	거래	거래 카테고리 변경	path: transactionId, body: categoryId	transactionId, categoryId, updated
GET	/api/v1/financial-events	예정 이벤트	기간 내 예정 입출금 조회	query: from, to, direction, eventType, status	events
POST	/api/v1/financial-events	예정 이벤트	예정 금융 이벤트 등록	body: eventDate, title, amount, direction, eventType, fixed	eventId, eventDate, amount, status
PATCH	/api/v1/financial-events/{eventId}	예정 이벤트	예정 금융 이벤트 수정	path: eventId, body: partial fields	eventId, updated
DELETE	/api/v1/financial-events/{eventId}	예정 이벤트	예정 금융 이벤트 취소	path: eventId	eventId, status=CANCELED
POST	/api/v1/recurring-rules	반복 규칙	월 반복 규칙 생성	body: title, recurrenceType, dayOfMonth, startDate, amount, eventType, direction	recurringRuleId, generatedEvents
GET	/api/v1/recurring-rules	반복 규칙	반복 규칙 목록 조회	none	rules
PATCH	/api/v1/recurring-rules/{id}	반복 규칙	반복 규칙 수정	path: id, body: partial fields	recurringRuleId, active
DELETE	/api/v1/recurring-rules/{id}	반복 규칙	반복 규칙 비활성화	path: id	recurringRuleId, active=false
GET	/api/v1/forecasts	예측	잔액 예측 타임라인 조회	query: from, to	timeline, minimumBalanceDate, weather
GET	/api/v1/dashboard	대시보드	홈 대시보드 조회	query: baseDate, targetDate optional	currentBalance, availableFunds, weather, risks, nextEvents
POST	/api/v1/available-funds/simulations	시뮬레이션	추가 지출 영향 계산	body: spendingDate, amount, targetDate, title	before, after, delta, weatherChange
POST	/api/v1/agent/conversations	AI Agent	AI 대화 세션 생성	body: title optional	conversationId

Method	Path	영역	목적	요청	응답
POST	/api/v1/agent/chat	AI Agent	AI 질문 답변 및 메시지 저장	body: conversationId, message	messageId, answer, sources
GET	/api/v1/agent/conversations/{id}/messages	AI Agent	대화 메시지 조회	path: id	messages
GET	/api/v1/agent/suggestions	AI Agent	추천 질문 조회	none	suggestions
POST	/api/v1/agent/analyze-spending	AI Agent	소비 패턴 분석	none	month, categories, totalExpense
POST	/api/v1/agent/detect-recurring	AI Agent	반복 지출 후보 탐지	none	candidates
POST	/api/v1/agent/recommend-actions	AI Agent	행동 추천	none	actions, reason
GET	/api/v1/agent/evidence/{messageId}	AI Agent	AI 답변 근거 조회	path: messageId	sources, toolSnapshot
GET	/api/v1/budgets/monthly	예산	월 예산 조회	none	month, categoryLimits, totalLimit
PUT	/api/v1/budgets/monthly	예산	월 예산 저장	body: month, categoryLimits, totalLimit	month, categoryLimits, totalLimit
GET	/api/v1/weather-statuses	안내	자금 날씨 상태 안내	none	statuses
GET	/api/v1/forecast-alerts	알림	예측 위험 알림 조회	none	alerts
GET	/actuator/health	운영	애플리케이션 상태 확인	none	status
GET	/actuator/metrics	운영	Micrometer 메트릭 목록/값 조회	metric name optional	names or measurements
GET	/actuator/prometheus	운영	Prometheus scrape endpoint	none	Prometheus text format

4. 공통 규칙

요청은 JSON 본문과 ISO 날짜 형식을 기본으로 사용합니다. 월은 yyyy-MM, 일자는 yyyy-MM-dd 형식입니다. 요청 값 검증 실패, 소유권 불일치, 존재하지 않는 리소스는 공통 에러 응답으로 반환됩니다.

항목	형식
Authorization	Bearer
대체 사용자 지정	X-User-Id: 1 또는 ?userId=1
성공 응답	각 API별 JSON 객체 또는 목록
에러 응답	{"timestamp": "...", "status": 400, "code": "VALIDATION_ERROR", "message": "요청 값이 올바르지 않습니다.", "path": "...", "fields": {...}}

5. 상세 API 설명

인증

API	내용	요청 예시	응답 예시
POST /api/v1/auth/dev-token	사용자 ID를 검증한 뒤 HMAC 기반 Bearer 토큰을 발급합니다. 실제 외부 로그인 연동 전까지 프론트와 API 테스트에서 사용할 수 있습니다.	{ "userId": 1 }	{ "tokenType": "Bearer", "accessToken": "" }

사용자/기초 데이터

API	내용	요청 예시	응답 예시
GET /api/v1/users/me	현재 사용자 컨텍스트를 확인합니다. Authorization, X-User-Id, userId query 순서로 사용자를 해석합니다.	query: userId=1	{ "userId": 1, "name": "Demo User", "baseMonth": "2026-09", "status": "ACTIVE" }
POST /api/v1/dev/seed	데모 사용자와 계좌, 카드, 카테고리, 거래, 이벤트, 예산 데이터를 생성합니다.	{ "userId": 1, "baseMonth": "2026-09", "reset": true }	{ "seeded": true, "userId": 1 }
GET /api/v1/accounts	자산 포함 계좌만 보거나 전체 계좌를 볼 수 있습니다.	query: includedOnly=true	{ "totalBalance": 2450000, "accounts": [...] }
GET /api/v1/cards	결제일 기반 이벤트와 연결되는 카드 정보를 조회합니다.	query: active=true	{ "cards": [...] }
GET /api/v1/categories	거래 분류, 예산 한도, 소비 분석에 공통으로 쓰는 카테고리 목록입니다.	query: categoryType=EXPENSE	{ "categories": [...] }

거래/예정 이벤트/반복 규칙

API	내용	요청 예시	응답 예시
GET /api/v1/transactions	월 단위 거래를 페이지네이션으로 조회하고 키워드, 유형, 카테고리 필터를 적용합니다.	query: month=2026-09&page=0&size=20	{ "items": [...], "totalElements": 10 }
PATCH /api/v1/transactions/{transactionId}/category	잘못 분류된 거래 카테고리를 사용자 소유 범위 안에서 수정합니다.	{ "categoryId": 3 }	{ "updated": true }
GET /api/v1/financial-events	급여, 카드 결제, 구독료처럼 미래 잔액에 영향을 주는 예정 이벤트를 조회합니다.	query: from=2026-09-01&to=2026-09-30	{ "events": [...] }
POST /api/v1/financial-events	단건 예정 입금/지출 이벤트를 생성합니다.	{ "eventDate": "2026-09-25", "title": "카드 결제", "amount": 320000, "direction": "OUTFLOW", "eventType": "CARD_PAYMENT", "fixed": true }	{ "eventId": 10, "status": "SCHEDULED" }

API	내용	요청 예시	응답 예시
PATCH /api/v1/financial-events/{eventId}	일자, 제목, 금액, 상태, 고정 지출 여부를 부분 수정합니다.	{"amount": 350000, "status": "SCHEDULED"}	{"updated": true}
DELETE /api/v1/financial-events/{eventId}	데이터 보존을 위해 실제 삭제 대신 CANCELED 상태로 변경합니다.	path: eventId	{"status": "CANCELED"}
POST /api/v1/recurring-rules	월 반복 규칙을 만들고 해당 월 예정 이벤트를 생성합니다.	{"title": "넷플릭스", "recurrenceType": "MONTHLY", "dayOfMonth": 10, "startDate": "2026-09-01", "amount": 17000, "eventType": "SUBSCRIPTION", "direction": "OUTFLOW"}	{"recurringRuleId": 1, "generatedEvents": 1}
GET/PATCH/DELETE /api/v1/recurring-rules	반복 규칙 목록 조회, 부분 수정, 비활성화를 제공합니다.	path/body by method	{"rules": [...]}

예측/대시보드/예산

API	내용	요청 예시	응답 예시
GET /api/v1/forecasts	일자별 예정 이벤트를 반영해 예상 잔액 타임라인과 최저 잔액일을 계산합니다.	query: from=2026-09-01&to=2026-09-30	{"timeline": [...], "minimumBalanceDate": "2026-09-25", "weather": "CLOUDY"}
GET /api/v1/dashboard	홈 화면에 필요한 현재 잔액, 고정 지출, 사용 가능 자금, 위험 요약, 다음 이벤트를 한 번에 반환합니다.	query: baseDate, targetDate optional	{"currentBalance": 2450000, "availableFunds": 1200000, "weather": "SUNNY"}
POST /api/v1/available-funds/simulations	추가 소비가 자금 날씨와 사용 가능 자기에 미치는 변화를 계산합니다.	{"spendingDate": "2026-09-15", "amount": 80000, "targetDate": "2026-09-30", "title": "외식"}	{"delta": -80000, "weatherChange": false}
GET/PUT /api/v1/budgets/monthly	월 예산과 카테고리별 한도를 조회하거나 저장합니다.	{"month": "2026-09", "categoryLimits": {"식비": 400000}, "totalLimit": 1200000}	{"month": "2026-09", "totalLimit": 1200000}

AI Agent

API	내용	요청 예시	응답 예시
POST /api/v1/agent/conversations	대화 세션을 만들고 이후 메시지를 conversationId에 연결합니다.	{"title": "이번 달 소비 점검"}	{"conversationId": 1}
POST /api/v1/agent/chat	대시보드, 예측, 이벤트 스냅샷을 근거로 답변을 생성하고 메시지를 저장합니다.	{"conversationId": 1, "message": "이번 달 괜찮아?"}	{"messageId": 2, "answer": "...", "sources": [...]}
GET /api/v1/agent/conversations/{id}/messages	대화 내 사용자/AI 메시지를 시간순으로 조회합니다.	path: id	{"messages": [...]}
GET /api/v1/agent/suggestions	홈 또는 AI 화면에 노출할 추천 질문 칩을 반환합니다.	none	{"suggestions": [...]}
POST /api/v1/agent/analyze-spending	현재 기준 월 지출을 카테고리별로 집계합니다.	none	{"categories": [...], "totalExpense": 830000}
POST /api/v1/agent/detect-recurring	활성 반복 규칙을 반복 지출 후보로 반환합니다.	none	{"candidates": [...]}
POST /api/v1/agent/recommend-actions	현재 자금 날씨와 위험 요약 기반 행동을 추천합니다.	none	{"actions": [...]}
GET /api/v1/agent/evidence/{messageId}	AI 답변에 사용한 출처와 도구 스냅샷을 조회합니다.	path: messageId	{"sources": [...], "toolSnapshot": {...}}

운영/모니터링

API	내용	요청 예시	응답 예시
GET /actuator/health	서버 상태와 readiness/liveness probe 확인에 사용합니다.	none	{"status": "UP"}
GET /actuator/metrics	JVM, HTTP, DB 커넥션 등 Micrometer 메트릭을 조회합니다.	metric name optional	{"names": [...]}
GET /actuator/prometheus	Prometheus가 scrape할 수 있는 텍스트 포맷 메트릭을 제공합니다.	none	Prometheus text format
api_request_logs	업무 API 요청의 userId, method, path, status, durationMs, clientIp, userAgent, createdAt을 저장합니다.	Flyway V2 table	운영 감사와 장애 분석에 사용

6. 내부 개선 완료 항목

항목	구현 내용	확인 방법
Flyway V2 이후 마이그레이션	V2_observability.sql로 api_request_logs 테이블과 user_id, created_at, path 인덱스를 추가했습니다.	테스트 부팅 시 V1, V2 migration 적용
Swagger 설명 보강	인증 API와 업무 API 전체에 정상 한국어 Tag/Operation 설명을 적용했습니다.	Swagger UI /swagger-ui/index.html
계산 로직 테스트 세분화	잔액 합산, 고정 지출 필터링, 날씨 등급, 최저 잔액일, 기간 검증을 단위 테스트로 분리했습니다.	ForecastServiceTest 5개
운영 로그/모니터링	Actuator health/metrics/prometheus와 API 요청 로그 필터 및 저장소를 추가했습니다.	/actuator/health, /actuator/prometheus

7. 외부 키가 필요한 영역

항목	설명
실제 AI 모델 호출	AI_PROVIDER=openai-compatible, AI_API_KEY, AI_MODEL 설정이 필요합니다. 키가 없으면 rule-based 답변으로 동작합니다.
파인튜닝/별도 모델 학습	학습 데이터셋, 모델 제공자, 비용/보안 정책 확정이 필요합니다. 현재 백엔드는 모델 교체 가능한 어댑터까지 구현했습니다.